

BITÁCORA DE DESARROLLO Y OPTIMIZACIÓN (L-IA v3.2.1)

DOCUMENTO CONSOLIDADO DE LA FASE 8

Punto estable de la nueva interfaz · Resumen de avance y registro detallado de sesiones

Fecha de consolidación	29 de septiembre de 2026
Tipo de documento	Documento unificado: resumen de avance (Parte I) y bitácoras de sesión (Parte II)
Estado	Punto estable consolidado. No constituye un cierre de fase: la interfaz permanece abierta a mejoras.
Alcance	Fase 8 (8.1 – 8.5), arquitectura multi-sesión y gestión de sesiones
Periodo cubierto	20 al 29 de septiembre de 2026
Desarrollador	Alejandro David Larco Tenelema
Entorno	Tauri v2 / React (Frontend) + FastAPI (Backend)
Hardware base	Intel Core i5-13420H RTX 4050 (6 GB VRAM) 16 GB DDR5 RAM

Contenido

Parte I	Resumen consolidado: propósito, línea de tiempo, estado de componentes, contrato frontend–backend, métricas, pendientes, verificación y próximos pasos	Secciones 1 – 10
Parte II	Registro detallado de sesiones: nueve bitácoras con formato uniforme (objetivo, cambios, incidencias, pendientes derivados)	Sesiones 1 – 9
Anexo A	Registro de decisiones técnicas y de diseño	Tabla A-1

1. Propósito y alcance

Este documento reúne en una sola referencia las bitácoras de sesión de la Fase 8 (20 al 28 de septiembre de 2026) y los registros posteriores de arquitectura multi-sesión y gestión de sesiones. Establece qué quedó integrado en la nueva interfaz de L-IA, qué se considera estable a partir de este punto y qué permanece pendiente.

Sirve como punto de retoma del trabajo sin necesidad de reconstruir el historial sesión por sesión. La Parte I ofrece la visión de conjunto; la Parte II conserva el detalle técnico de cada sesión con un formato uniforme, y cada hito de la línea de tiempo remite a su sesión correspondiente.

2. Resumen ejecutivo

La nueva interfaz pasó de ser una ventana de chat con respuestas en bloque en Tkinter a un HUD reactivo con Tauri+React. Las respuestas llegan en streaming (SSE); el avatar holográfico 2.5D reacciona al estado del sistema con un costo de 0 MB de VRAM; las generaciones pueden abortarse desde la interfaz sin dejar hilos activos en la GPU; y los documentos se ingieren en un pipeline dual (memoria vectorial + workspace activo). La última pieza integrada fue la arquitectura multi-sesión, que aísla historial y workspace por pestaña.

A partir de este punto, frontend y backend comparten un contrato estable de endpoints y eventos (sección 5). El trabajo pendiente corresponde a refinamiento y nuevas capacidades, no a reestructuración.

3. Línea de tiempo de la integración

Fecha	Fase	Hito principal	Sesión
20 sep 2026	8 / 8.1	Streaming SSE en vivo (TTFT de 9.13 s a 0.47 s), efecto Typewriter, badges de motor, carga diferida de modelos y estabilización de VRAM.	1
21 sep 2026	8 (red y errores)	Bifurcación de renderizado para bloques masivos, corrección del modelo Nube Pro (error 404), keep-alive visual y reintentos ante desconexiones.	2
21 sep 2026	8.2	Kill Switch (threading.Event + /cancelar + AbortController), textarea multilínea y estado visual de mensaje abortado.	3
22 sep 2026	8.3	Avatar holográfico 2.5D, parpadeo autónomo y expresiones según la fase del sistema.	4

Fecha	Fase	Hito principal	Sesión
22 sep 2026	8.4	Parallax ocular, ciclo de inactividad, micro-expresiones tácticas y renderizador Markdown con resaltado y botón Copiar.	5
22 sep 2026	8.5	Ingesta híbrida RAG + Workspace activo, badge [TRABAJANDO EN] y Semáforo v3.	6
28 sep 2026	8.5 (consolidación)	Función maestra de ingesta, enrutador semántico natural, blindaje del prompt y sincronización reactiva.	7
Sep 2026 (sin fecha exacta)	Consolidación sensorial	Arquitectura multi-sesión, panel de pestañas, autonombre de sesiones e interceptor global de Drag & Drop.	8
Sep 2026 (sin fecha exacta)	Memoria y UI nativa	Restauración de rutas PUT/DELETE de sesiones, corrección de peticiones en Tauri y ventana sin marco nativo.	9

4. Estado consolidado de la interfaz

El estado de cada componente se toma de lo reportado en las bitácoras; la verificación final se presenta en la sección 8.

Componente	Descripción	Estado
Streaming y Typewriter	/chat con StreamingResponse; búfer de 15 ms carácter por carácter; los bloques de más de 50 caracteres se inyectan de una sola vez.	Consolidado
Etiquetas de motor	Badges [LOCAL] (cian), [NUBE] (morado) y [DOLPHIN] (rojo táctico).	Consolidado
Interruptor de cancelación	Botón Detener: AbortController en el fetch y /cancelar en el backend; la burbuja abortada baja al 35 % de opacidad con borde rojo segmentado.	Consolidado
Entrada de comandos	Textarea de 42 a 100 px; Enter envía y Shift + Enter agrega salto de línea; scrollbar personalizada en cian.	Consolidado
Núcleo holográfico 2.5D	Rostro con framer-motion y CSS puro (0 MB de VRAM): parpadeo, estados procesando / escribiendo / error, seguimiento del cursor y ciclo de inactividad hasta el estado durmiendo.	Consolidado
Micro-expresiones	Lentes holográficos en RAG, clon lateral en comandos Git e inclinación de 15° ante interrogación o ambigüedad.	Consolidado

Componente	Descripción	Estado
Renderizador Markdown	react-syntax-highlighter con tema vscDarkPlus, botón de copiado con navigator.clipboard y protección de código inline.	Consolidado
Ingesta y Workspace	Ingesta hacia /ingestar desde la app mediante el botón de búsqueda de archivos o la mención del archivo: vectorización en ChromaDB, anclaje en SQLite y micro-resumen de 25 palabras con Gemma 2; limpieza con /workspace/limpiar.	Consolidado
Multi-sesión	Tabla sesiones_chat, parámetro sesion_id en todo el flujo, panel lateral con GET /sesiones y autonombre de sesiones en hilo aparte.	Consolidado
Gestión de sesiones	Edición y borrado de sesiones mediante PUT y DELETE /sesiones/{id}; sin dependencia de diálogos nativos del navegador.	Consolidado
Ventana nativa	Marco del sistema operativo eliminado (decorations: false) y zona superior arrastrable con data-tauri-drag-region.	Consolidado
Drag & Drop global	Listeners en window que anulan el comportamiento nativo de WebView2 y evitan el secuestro del evento onDrop. En la versión app se decidió no usar el arrastre como vía de ingesta; queda reservado para la versión web.	Diferido (versión web)

5. Contrato frontend – backend

Puntos de contacto que se consideran estables desde este punto:

Endpoint / parámetro	Función
/chat	Respuesta en streaming SSE; emite un evento de tipo fin con el nombre del workspace al terminar el texto.
/cancelar	Activa la bandera global que rompe el bucle de generación token a token.
/ingestar	Ejecuta cerebro.asimilar_documento_maestro (extracción, ChromaDB, SQLite y micro-resumen).
/workspace/limpiar	Purga el registro del workspace activo.
GET /sesiones	Lista las sesiones para el panel de pestañas.
PUT /sesiones/{id}	Actualiza (renombra) una sesión existente.
DELETE /sesiones/{id}	Elimina una sesión.

Endpoint / parámetro	Función
sesion_id	Viaja en las llamadas de chat e ingesta para mantener el aislamiento de memoria por sesión.

6. Métricas de referencia

Métrica	Valor
Tiempo hasta el primer token (TTFT)	9.13 s → 0.47 s (reducción del 94 %)
Velocidad de generación local (Gemma 2)	12.5 – 14 t/s
Impacto del avatar en VRAM	0 MB
Costo acumulado de desarrollo	1.10 USD hasta el 29 de septiembre (0.38 USD al cierre del 21 de septiembre)
Consumo en la nube el 21 sep	17 peticiones Flash y 2 Pro (cuotas diarias de 10K y 250)
Privacidad de datos en la nube	Google AI Studio sin registros de transacciones ("No logs found") en la auditoría del 21 sep
Release de escritorio	Tauri v0.1.0 compilada con éxito el 20 sep; el build de Vite alerta por chunks mayores a 500 kB

7. Pendientes, deuda técnica y riesgos

7.1 Registro de pendientes

Elemento	Origen	Estado
Kill Switch para interrumpir la generación	Fase 8.1	Resuelto en 8.2
Respuesta vacía / error 404 en la ruta Nube Pro	Fase 8.1	Resuelto (21 sep)
Botón de copiar código en Markdown	Fase 8.3	Resuelto en 8.4
Conflicto de Drag & Drop (onDragLeave fantasma)	Fase 8.5	Resuelto (interceptor global)
Persistencia del historial visual al iniciar	Fase 8.1	Resuelto
Pérdida de saltos de línea (lnln) en el stream de la Nube	Fase 8.3	Resuelto
Refinamiento UX del chat (interlineado, scroll, colores de botones)	Fase 8.3	Resuelto

Elemento	Origen	Estado
Prefijo “L-IA:” en dolphin-mistral (System Prompt de su ruta)	Fase 8.1	Resuelto
Endpoints PUT/DELETE de sesiones ausentes tras restaurar backup de api.py	Sesión 9	Resuelto
Calibración de consultas RAG exploratorias	Fase 8.5	Replanteado para la Fase 9
Sincronización labial con Edge-TTS	Fase 8.4	Planificado (Fase 10)
Estado procesando_sistema (cursor >_ en lugar del rostro)	Fase 8.4	Pendiente (sin constancia de implementación)
Ingesta por arrastre en la versión app	Fase 8.5	Diferido (versión web)

Los elementos marcados como “Pendiente” pueden estar resueltos en el código; simplemente no quedó constancia en las bitácoras.

7.2 Riesgos y observaciones

- **Identificadores de modelos en preview.** Los identificadores de Gemini en preview pueden cambiar; el error 404 ya ocurrió una vez por esta causa.
- **Tiempos de la ruta Pro.** La llamada síncrona a Pro depende del mensaje de keep-alive para no cerrar el canal SSE; conviene vigilar los tiempos si el modelo tarda más.
- **Peso del bundle.** El bundle de Vite creció con el resaltado de sintaxis; no afecta la fluidez en Tauri, pero conviene considerar la división de código (code splitting).
- **Respalos de `api.py`.** Restaurar un backup hizo desaparecer rutas ya implementadas (Sesión 9); se recomienda validar el listado de endpoints tras cualquier restauración.

8. Verificación del punto estable

Pruebas de humo realizadas antes de tomar este punto como referencia oficial:

Prueba	Resultado	Observaciones
Backend FastAPI y app Tauri se comunican correctamente	Aprobado	—
Streaming operativo en rutas Local, Nube Flash y Nube Pro	Aprobado	—
El botón Detener corta la generación y libera la GPU	Aprobado	—

Prueba	Resultado	Observaciones
Arrastrar un archivo fija el workspace y se puede consultar sobre él	Descartado en versión app	Se decidió usar exclusivamente el botón de búsqueda de archivos o el guardado mediante mención. El arrastre se deja para la versión web y se pospone a versiones futuras.
Crear y alternar sesiones sin mezclar historial ni workspace	Aprobado	—
Bloque de código con resaltado y botón Copiar	Aprobado	—
Build de release de Tauri sin errores	Aprobado	—
Repositorio sincronizado y punto estable etiquetado en Git	Aprobado	—

9. Próximos pasos

1. Cerrar los pendientes abiertos de la sección 7 (en particular el estado `procesando_sistema`).
2. Iniciar la planificación de generación y refactorización masiva de código con Gemini Pro.
3. Preparar el terreno para la Memoria de Entidades sobre la base multi-sesión.
4. Fase 9: reactivar Edge-TTS y replantear la calibración de consultas RAG exploratorias.
5. Fase 10: sincronizar la boca del avatar (barras ecualizadoras) con los picos RMS del audio.

10. Conclusión

La integración de la nueva interfaz queda consolidada en un punto estable: streaming, cancelación, avatar reactivo, ingesta híbrida y multi-sesión funcionan sobre un contrato claro entre Tauri/React y FastAPI. No se cierra la fase, pero el siguiente trabajo parte de esta base y no de cero.

PARTE II · REGISTRO DETALLADO DE SESIONES

Cada sesión sigue la misma estructura: ficha de identificación, objetivo y resultado, cambios implementados, incidencias con su resolución y pendientes derivados con su estado actual. Las sesiones se presentan en orden cronológico; las dos últimas carecen de fecha exacta en su registro original.

Fecha	20 de septiembre de 2026	Estado	Cerrado
Módulo	Fase 8 / 8.1 (UX/UI, fluidez cognitiva y telemetría de hardware)		

Objetivo y resultado

Erradicar la latencia del sistema y transformar la entrega de respuestas de bloques estáticos a un flujo dinámico en tiempo real. Se redujo el Tiempo Hasta el Primer Token (TTFT) en un **94 %** (de 9.13 s a 0.47 s) mediante refactorización arquitectónica y un canal de Server-Sent Events (SSE) que da a la interfaz una escritura fluida.

Cambios implementados

Área	Cambio implementado
Streaming (backend)	Reescritura del endpoint /chat de FastAPI: las respuestas JSON tradicionales se reemplazan por un StreamingResponse apoyado en queue.Queue.
Streaming (frontend)	Búfer de interpolación asíncrona (micro-pausa de 15 ms) que procesa las ráfagas TCP y renderiza el texto carácter por carácter (efecto Typewriter), respetando el formateo de react-markdown.
Telemetría aislada	Script diagnostico_local.py para medir el rendimiento puro de la RTX 4050 frente al pipeline completo del cerebro.
Arranque	Patrón de carga diferida (lazy loading) para el modelo acústico (faster-whisper en core.voz) y la base vectorial (ChromaDB en MemoriaRAG): se cargan en memoria únicamente cuando el flujo lo requiere. Se eliminó el cuello de botella causado por la carga secuencial de modelos masivos.
Estabilización de VRAM	Se eliminó el parámetro rígido num_gpu: 33 en la API de Ollama y se delegó el cálculo al motor nativo. Esto detuvo el desbordamiento hacia la RAM y estabilizó Gemma 2 en 12.5 – 14 t/s, el límite óptimo para la GPU actual.
Etiquetas HUD	Badges dinámicos del motor de inferencia: [LOCAL] (cian), [NUBE] (morado) y [DOLPHIN] (rojo táctico).
Anclaje de contexto	Corrección del anclaje visual (el sistema mantenía el archivo pegado al chat): FastAPI lee el workspace_activo exacto desde SQLite (database.obtener_hecho) en cada turno.

Incidencias y resoluciones

Incidencia	Causa / síntoma	Resolución
Bloqueo de compilación en Rust (os error 5)	Acceso denegado en npx tauri build; el binario app.exe persistía en segundo plano.	Se forzó la detención del proceso. La release v0.1.0 compiló con éxito.

Incidencia	Causa / síntoma	Resolución
Prefijos teatrales en dolphin-mistral	El modelo enrutó y transmitió correctamente, pero imprimía el prefijo “L-IA:”.	Requería ajuste en el System Prompt de su ruta. Resuelto posteriormente.

Pendientes derivados

Pendiente derivado	Estado actual
Auditoría de la ruta Nube Pro (respuesta vacía con gemini-3.1-pro tras el streaming)	Resuelto (21 sep)
Mecanismo de aborto (Kill Switch)	Resuelto en 8.2
Persistencia del historial visual al iniciar la app	Resuelto

Sesión 2 · 21 sep 2026 — Refinamiento de red y manejo de errores de API

Fecha	21 de septiembre de 2026 · cierre 15:31	Estado	Cerrado (repositorio sincronizado)
Módulo	Fase 8 (refinamiento de red y manejo de errores de API)		

Objetivo y resultado

Depurar el comportamiento asíncrono de la interfaz ante respuestas estáticas, corregir el enrutamiento hacia modelos obsoletos de la Nube y dotar al sistema de resiliencia frente a micro-cortes de conexión. El ecosistema híbrido alcanzó una estabilidad alta combinando el procesamiento local con la API de Google, con un costo operativo mínimo.

Incidencias y resoluciones

Incidencia	Causa / síntoma	Resolución
Saturación del búfer con bloques masivos	El efecto Typewriter (App.jsx) colapsaba al animar reportes estáticos extensos, como los de las herramientas Git o RAG.	Bifurcación de renderizado: los fragmentos de más de 50 caracteres omiten la pausa de 15 ms y se inyectan de una sola vez; el flujo en vivo de Ollama/Gemini conserva la animación. En el backend se inyectó <code>callback_stream</code> antes de los return directos en <code>_ejecutar_guardado_git</code> y <code>_procesar_ingesta_documento</code> (cerebro.py).
Error 404 en la Nube Pro	Los identificadores <code>gemini-3.1-pro</code> y <code>gemini-2.5-pro</code> quedaron obsoletos en la API v1beta.	Se actualizó <code>MODELO_NUBE_PRO</code> a <code>gemini-3.1-pro-preview</code> , restableciendo el acceso al motor de razonamiento avanzado.

Incidencia	Causa / síntoma	Resolución
Cierre prematuro del socket SSE y errores silenciosos (Nube Pro)	El tiempo de generación de Pro cerraba el canal SSE en el frontend y los errores de API se perdían en el backend.	Keep-alive visual: antes de la llamada síncrona se envía el mensaje “(Analizando arquitectura en la Nube Pro, un momento...)”. Todos los bloques except de <code>responder_con_nube</code> transmiten el error mediante <code>callback_stream</code> antes de abortar.
Desconexión por corte eléctrico	Fallo de socket (Server disconnected) que L-IA analizó automáticamente.	Las desconexiones abruptas (por ejemplo, Connection reset, 104) se integraron a la lógica de errores transitorios de <code>_manejar_error_nube</code> , con reintentos automáticos.

Auditoría de costos y telemetría

Indicador	Valor registrado
Costo acumulado del periodo	0.38 USD
Gasto del 21 de septiembre	0.036 USD
Peticiones a la nube	17 Flash y 2 Pro; el enrutamiento local absorbió casi toda la carga transaccional
Cuotas diarias	10K (Flash) y 250 (Pro); consumo muy por debajo del límite
Privacidad	Google AI Studio no muestra registros de transacciones (“No logs found”): el código local y las rutas de directorios no se almacenan en datasets remotos

Conclusión de la sesión

El núcleo asíncrono y el gestor de inferencia quedan formalmente cerrados. El sistema enruta hardware, código, APIs y comandos visuales bajo una arquitectura de costo cercano a cero.

Sesión 3 · 21 sep 2026 — Kill Switch y entrada multilínea (Fase 8.2)

Fecha	21 de septiembre de 2026	Estado	Cerrado
Módulo	Fase 8.2 (mecanismo de interrupción y refinamiento UX/UI)		

Objetivo y resultado

Integrar el mecanismo de interrupción de inferencia (Kill Switch) y modernizar la captura de comandos. Se logró abortar procesos pesados sin dejar hilos activos consumiendo la GPU, y admitir directivas multilínea estructuradas.

Cambios implementados

Área	Cambio implementado
Cancelación (backend)	core/cerebro.py y api.py: un <code>threading.Event()</code> actúa como bandera global. El nuevo endpoint <code>/cancelar</code> la activa y fuerza la ruptura inmediata del bucle <code>token</code> a <code>token (_generar_respuesta_con_voz)</code> , liberando la RTX 4050.
Cancelación (frontend)	App.jsx: la API <code>AbortController</code> se integra al <code>fetch</code> . Al pulsar “Detener” se corta la conexión SSE y se envía la señal asíncrona de aborto al núcleo de Python.
Entrada multilínea	El input de una sola línea se reemplaza por un textarea responsivo que respeta el diseño HUD: altura base de 42 px, con crecimiento hasta 100 px.
Atajos de teclado	Enter ejecuta el envío; Shift + Enter inserta un salto de línea.
Scrollbar	La barra estándar de Windows se sustituye por una custom-scrollbar en la paleta cian del núcleo.
Estados de fallo inducido	El bloque <code>catch</code> distingue caídas reales de abortos inducidos (<code>AbortError</code>). Los mensajes cancelados reciben <code>msg.cancelado</code> , que activa la clase mensaje-abortado: opacidad al 35 %, filtro de escala de grises y borde rojo segmentado.
Prevención de envío múltiple	El estado <code>cargando</code> bloquea nuevos envíos hasta que L-IA termine de procesar o sea abortada formalmente.

Siguientes directivas

Sincronizar el repositorio con los cambios finales del frontend e iniciar la planificación de generación y refactorización masiva de código con Gemini Pro (ver sección 9).

Sesión 4 · 22 sep 2026 — Avatar holográfico 2.5D (Fase 8.3)

Fecha	22 de septiembre de 2026	Estado	Hito visual completado; migración a nueva instancia de chat
Módulo	Fase 8.3 (interfaz holográfica 2.5D y reactividad facial)		

Objetivo y resultado

Sustituir la iconografía estática por un núcleo híbrido interactivo que combina los anillos orbitales con un avatar holográfico (falso 3D). La interfaz transmite reactividad mediante expresiones generadas matemáticamente, con un costo de **0 MB de VRAM**, protegiendo los recursos de la RTX 4050 para el LLM local. Además, L-IA superó una prueba de visión computacional al auditar su propio frontend y sugerir mejoras de UI.

Cambios implementados

Área	Cambio implementado
Avatar 2.5D	Se eliminaron los iconos estáticos de Lucide y se estructuró un rostro minimalista con framer-motion y CSS puro. Gradientes radiales oscuros en el centro del núcleo crean contraste para que la cara “brille” desde el interior.
Parpadeo autónomo	useEffect con intervalos aleatorios entre 2 y 6 segundos que simula un parpadeo natural en reposo.
Estado: procesando / RAG	Ojos redondos y movimiento lateral (escaneo).
Estado: escribiendo	Ojos achinados y boca con escala dinámica que simula el habla.
Estado: error	Inclinación asimétrica (mueca), temblor de posición (glitch) y color rojo de alerta.

Incidencias y resoluciones

Incidencia	Causa / síntoma	Resolución
Colapso total de la interfaz	Pantalla blanca/negra causada por variables huérfanas y llamadas a componentes inexistentes dentro del ciclo de vida del estado.	Se identificaron y eliminaron las referencias huérfanas; el renderizado se restableció.

Pendientes derivados

El análisis visual realizado por la propia L-IA fijó el objetivo de la siguiente sesión:

Pendiente derivado	Estado actual
Saneamiento del stream SSE de la Nube (pérdida de \n\n en los chunks de Gemini Pro)	Resuelto
Renderizador Markdown con botón “Copiar código”	Resuelto en 8.4
Refinamiento UX del chat (interlineado de listas, scroll, colores de botones sincronizados con el núcleo)	Resuelto

Sesión 5 · 22 sep 2026 — Comportamiento orgánico y Markdown avanzado (Fase 8.4)

Fecha	22 de septiembre de 2026	Estado	Cerrado
Módulo	Fase 8.4 (comportamiento orgánico 2.5D y renderizado Markdown avanzado)		

Objetivo y resultado

Dotar al núcleo holográfico de un comportamiento orgánico y reactivo a su entorno mediante una máquina de estados visuales basada en framer-motion y CSS puro, manteniendo el impacto en VRAM en 0 MB. En paralelo se resolvió la deuda técnica del renderizado de código con resaltado de sintaxis y copiado al portapapeles.

Cambios implementados

Área	Cambio implementado
Seguimiento ocular	Listener global de coordenadas del ratón (onMouseMove) que mapea el desplazamiento del cursor a la propiedad transform del rostro, logrando un efecto de profundidad (parallax 2.5D).
Ciclo de inactividad	Temporizador maestro que rastrea la falta de interacción: a los 3 s, miradas errantes aleatorias; a los 20 s, animación de bostezo (expansión y rotación asimétrica); a los 30 s, estado durmiendo con ojos cerrados y partículas “Zzz” generadas dinámicamente.
Micro-expresión: RAG	Al procesar documentos, el núcleo despliega lentes holográficos (backdrop-filter: blur) y reduce los ojos, simulando lectura profunda.
Micro-expresión: Git	Al invocar comandos de repositorio, el orbe genera una clonación lateral temporal con línea conectora, emulando la creación de un branch.
Micro-expresión: duda	La detección de signos de interrogación o palabras ambiguas inclina el núcleo 15° y altera la escala de un solo ojo (ceja analítica). Las micro-expresiones se activan en el frontend antes de que el servidor responda.
Resaltado de sintaxis	Integración de react-syntax-highlighter con el tema vscDarkPlus para PHP, C++, JavaScript y otros, acoplado a la paleta cian y negra del HUD.
Copiado rápido	Se sobrescribió el componente code de react-markdown. Si detecta un bloque multilínea (expresión regular y saltos de línea), genera un contenedor con la etiqueta del lenguaje y un botón que usa navigator.clipboard.
Código inline	Las variables sueltas (por ejemplo, \$mayor) se renderizan como insignias verdes simples, evitando el desbordamiento visual.

Estado del sistema y rendimiento

react-syntax-highlighter añadió peso al bundle de Vite en el build final (alertas de chunks > 500 kB), pero la fluidez en tiempo de ejecución bajo Tauri se mantiene intacta. El cómputo visual de las partículas CSS y las físicas spring es absorbido por la CPU.

Pendientes derivados

Pendiente derivado	Estado actual
Auditoría visual de terminal: estado procesando_sistema con cursor parpadeante >_ al ejecutar scripts de Python o diagnósticos de hardware	Pendiente (sin constancia de implementación)
Sincronización labial: barras ecualizadoras reactivas a picos RMS con Edge-TTS	Planificado (Fase 10)

Sesión 6 · 22 sep 2026 — Ingesta híbrida RAG + Workspace activo (Fase 8.5)

Fecha	22 de septiembre de 2026	Estado	Cerrado
Módulo	Fase 8.5 (ingesta híbrida RAG + Workspace activo)		

Objetivo y resultado

Fusionar la memoria a largo plazo con la atención a corto plazo: al incorporar un archivo, el backend ejecuta dos tareas en paralelo y la interfaz refleja el documento activo.

Cambios implementados

Área	Cambio implementado
Pipeline dual	El flujo de ingesta se enlazó con el endpoint /ingestar de FastAPI. Largo plazo (RAG): el archivo se vectoriza y almacena en ChromaDB para consultas históricas. Corto plazo (Workspace): el archivo se fija como entorno de trabajo activo en SQLite, permitiendo consultas inmediatas y contextuales.
Indicador visual	Badge dinámico [TRABAJANDO EN: archivo] con botón (X) para limpiar el workspace en cualquier momento mediante el nuevo endpoint /workspace/limpiar.
Enrutamiento inteligente	El Semáforo v3 distingue entre consultas generales de historial (RAG) y análisis profundo del documento activo (Workspace), asegurando la herramienta correcta para cada intención.

Nota de decisión

Esta sesión introdujo el arrastre de archivos (Drag & Drop) como vía de ingesta. Posteriormente, para la versión app se decidió usar exclusivamente el botón de búsqueda de archivos o el guardado mediante mención, reservando el arrastre para la versión web (ver sección 8 y Anexo A).

Sesión 7 · 28 sep 2026 — Foco de atención y enrutamiento semántico (Fase 8.5)

Fecha	28 de septiembre de 2026	Estado	Cerrado
Módulo	Fase 8.5 (ingesta híbrida, foco de atención y enrutamiento semántico)		

Objetivo y resultado

Resolver la brecha cognitiva entre la interfaz gráfica y el motor lógico respecto a la atención a corto plazo. La ingesta ejecuta un pipeline dual (memoria vectorial + entorno activo) que permite procesar documentos extensos en un contexto enfocado. Se eliminó la “amnesia” del LLM mediante directivas de cumplimiento obligatorio en el System Prompt y una expresión regular más flexible.

Cambios implementados

Área	Cambio implementado
Función maestra de ingesta	La lógica se desacopló del endpoint /ingestar y se centralizó en cerebro.asimilar_documento_maestro, que orquesta cuatro pasos: extracción de texto, vectorización en ChromaDB (largo plazo), anclaje de ruta en SQLite (foco activo) y micro-resumen sintético de 25 palabras con Gemma 2 local.
Enrutador semántico natural	Se reescribió PATRON_LECTURA_IMPLICITA para capturar lenguaje coloquial (“workspace”, “este proyecto”, “del archivo”), de modo que el usuario se refiera al documento fijado sin declarar rutas absolutas.
Lectura profunda a demanda	El sistema distingue entre usar el micro-resumen para respuestas rápidas y ejecutar una lectura completa del archivo físico, vía modelo en la nube, cuando se piden análisis exhaustivos (por ejemplo, resúmenes globales).
Blindaje del prompt	prompt_builder.py inyecta una directiva innegociable cuando existe un Workspace activo, neutralizando la tendencia del modelo (en especial con su personalidad sarcástica) a evadir consultas ambiguas o afirmar que desconoce el archivo anclado.
Sincronización reactiva	El endpoint /workspace/limpiar y el badge dinámico [TRABAJANDO EN: archivo] con control de cierre (X) purgan al instante el registro en SQLite y devuelven a L-IA al escaneo global.

Estado del sistema

La arquitectura de inyección de contexto inmediato es un éxito operativo: el usuario puede subir un archivo, convertirlo en el foco de la IA, dialogar sobre él sin pérdida de hilo y liberarlo a voluntad.

Pendientes derivados

Pendiente derivado	Estado actual
Conflicto de Drag & Drop (onDragLeave fantasma que cancela la subida al arrastrar sobre la interfaz)	Resuelto (interceptor global)

Pendiente derivado	Estado actual
Calibración de consultas RAG: evitar que consultas exploratorias globales (“¿de qué trataba el archivo?”) colapsen el buscador vectorial y orientarlo a la recuperación técnica de fragmentos	Replanteado para la Fase 9

Sesión 8 · 29 Sep 2026 — Consolidación sensorial y arquitectura multi-sesión

Fecha	29 Septiembre de 2026	Estado	Cerrado
Módulo	Fase de consolidación sensorial y expansión arquitectónica		

Objetivo y resultado

Migrar L-IA de un modelo de contexto único a una arquitectura multi-sesión con aislamiento de memoria (estilo ChatGPT/Claude), resolver bloqueos de interfaz y preparar el terreno para la Memoria de Entidades.

Cambios implementados · Backend

Área	Cambio implementado
Reestructuración de SQLite	database.py: creación de la tabla sesiones_chat para gestionar UUIDs independientes. Las variables workspace_activo, workspace_resumen y workspace_historial se desacoplaron de la memoria global y se anclaron a cada sesión.
Enrutamiento compartimentado	cerebro.py y api.py: inyección transversal del parámetro sesion_id. El gestor de herramientas, el RAG y las lecturas de entorno impactan únicamente el compartimento activo.
Aislamiento de prompts	prompt_builder.py: el ensamblador inyecta solo el historial y el documento activo de la pestaña en foco.
Autonombre de sesiones	Hilo en segundo plano (threading.Thread) que invoca a Gemma 2 para leer el primer mensaje del usuario y generar un título contextual corto, actualizando SQLite sin bloquear el streaming de la respuesta principal.

Cambios implementados · Frontend

Área	Cambio implementado
Gestor de pestañas	Panel lateral en React que consume GET /sesiones y permite alternar entre flujos de trabajo, destacando si una sesión inactiva tiene un archivo de proyecto anclado.
Sincronización reactiva	El SSE emite el nombre del workspace al finalizar la transmisión (evento fin). El badge verde neón del HUD reacciona de inmediato a lecturas de pantalla autónomas, sin recargar.

Área	Cambio implementado
Interceptor global de Drag & Drop	Listeners globales en window anulan el comportamiento nativo de WebView2 en Tauri, previniendo el secuestro del evento onDrop por el sistema operativo y eliminando parpadeos de CSS y bloqueos.

Sesión 9 · 29 Sep 2026 — Gestión de memoria y ventana nativa

Fecha	29 Septiembre de 2026	Estado	Cerrado
Módulo	Gestión de memoria y UI nativa (FastAPI/SQLite, React/Vite, Tauri)		

Objetivo y resultado

Restaurar el control total sobre los historiales de chat (edición y borrado) y modernizar el comportamiento de la ventana de la aplicación.

Incidencias y resoluciones

Incidencia	Causa / síntoma	Resolución
Errores 404 silenciosos en Uvicorn (backend)	Los endpoints de actualización y eliminación desaparecieron al restaurar un backup de seguridad de api.py.	Se reinyectaron las rutas PUT /sesiones/{id} y DELETE /sesiones/{id}, restableciendo la comunicación con la base de datos sin errores de sintaxis ni caídas del servidor.
Peticiones de red que no salían (frontend)	Tauri bloqueaba el popup nativo window.confirm, y un error de interpolación en las variables del fetch rompía la ruta.	Se eliminó la dependencia de alertas del navegador y se aplicó concatenación estricta (API + "/sesiones/" + id) para garantizar la entrega de la petición.

Cambios implementados · Interfaz (Tauri)

Área	Cambio implementado
Ventana sin marco	Se eliminó el marco predeterminado del sistema operativo ("decorations": false en tauri.conf.json) para un aspecto limpio.
Zona de arrastre	Se añadió un área superior invisible con el atributo data-tauri-drag-region, que permite arrastrar y posicionar L-IA en cualquier parte del monitor.

ANEXO A · REGISTRO DE DECISIONES TÉCNICAS

Decisiones de arquitectura y diseño que condicionan el trabajo futuro, con su motivo y la sesión donde se documentaron.

Decisión	Motivo	Sesión
Avatar en CSS puro y framer-motion	Mantener el costo en VRAM en 0 MB para reservar la RTX 4050 al LLM local.	4, 5
Carga diferida de faster-whisper y ChromaDB	Eliminar el cuello de botella de arranque causado por la carga secuencial de modelos masivos.	1
Delegar el cálculo de capas GPU a Ollama (sin num_gpu)	Evitar el desbordamiento a RAM y estabilizar la generación local en 12.5 – 14 t/s.	1
Bifurcación de renderizado (umbral de 50 caracteres)	Evitar el colapso del efecto Typewriter con bloques estáticos extensos.	2
Keep-alive visual y reintentos ante errores transitorios	Mantener abierto el canal SSE en llamadas largas a Pro y tolerar micro-cortes de red.	2
Cancelación mediante bandera global (threading.Event)	Interrumpir el bucle token a token y liberar la GPU sin hilos residuales.	3
Ingesta híbrida (RAG + Workspace)	Combinar consulta histórica (largo plazo) con foco inmediato sobre el documento activo (corto plazo).	6, 7
Aislamiento de historial y workspace por sesión	Evitar mezclas de contexto entre pestañas y preparar la Memoria de Entidades.	8
Ingesta en la app por botón de búsqueda o mención; arrastre reservado a la versión web	Descartar el arrastre como vía de ingesta en Tauri; se mantiene el interceptor global para evitar el secuestro de eventos.	6, 8; sección 8
Sin diálogos nativos del navegador (window.confirm)	Tauri los bloquea y las peticiones no llegaban a salir.	9